

AN INNOVATIVE, PROGRAMMABLE ARCHITECTURE FOR ULTRA-LOW POWER MOTION ESTIMATION IN REDUCED MEMORY MPEG-4 ENCODER

Daniele Alfonso, Fabrizio Rovati, Danilo Pau and Luca Celetto
Advanced System Technology Laboratories, STMicroelectronics, Agrate Brianza, Italy

ABSTRACT

This paper describes an ultra-low power, cache based, and programmable motion estimator with memory reduction for MPEG-4 video encoding. It exploits a low complexity motion estimation algorithm, achieving a quality comparable or superior to the Full Search approach with only 1% of the computation and the power consumption.

1. INTRODUCTION

Next generation mobile devices will offer multimedia capabilities, providing to the end user different kinds of new services, like multimedia messaging and videoconferencing. In fact, the 3GPP standardization committee for the third generation mobile phones recently chose the MPEG-4 Simple Profile [1] for video streaming in the wireless environment [2].

The designer who wishes to develop an efficient video encoding system for mobile devices has to deal with two main problems. The first one is the low bandwidth available, in the order of 64 kb/s to 384 kb/s: high performance algorithms must then be used to achieve the highest compression without compromising the quality of the encoded video sequences. The second one is the lower-power issue: being battery powered, these devices must do a smart use of the available energy resources.

Being the motion estimation one of the most computation intensive tasks in the video encoding, it is crucial to find an efficient hardware implementation.

Clearly, the classical Full Search Block Matching approach is far too complex for a mobile device, even when using a optimized architecture [3] [4] [5] [6] [7]. Different techniques are known to reduce the computational complexity of the motion estimation process, mainly based on the decimation of the motion field, i.e. Three-Step [8] [9] [10] and Four-Step [11] search, Logarithmic Search [12], Hierarchical Search [13], Window Follower [14], Diamond Search [15] and other more advanced techniques like the Rate/Distortion optimizations [16] or the fuzzy logic [17] and genetic based algorithms [18]. Other techniques tends to decimate the pixel domain, computing an approximate value of the matching error [19] [20].

Among the various possibilities, the spatial-temporal correlation algorithms [21] [22] proved their excellent

performances in terms of overall video quality versus the number of operations performed, and they are suitable for efficient hardware implementations [22] [23] [24] [25].

In this paper, a novel motion estimator subsystem for Simple Profile MPEG-4 video encoding is proposed. It exploits a low-complexity motion estimation algorithm, achieving a video quality comparable to the Full Search approach with only a very small fraction of the computation.

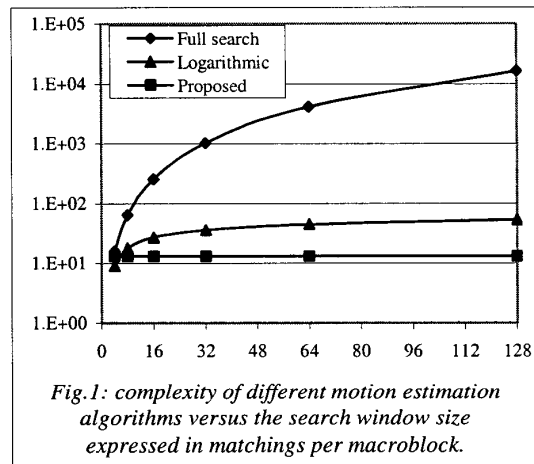


Fig.1: complexity of different motion estimation algorithms versus the search window size expressed in matchings per macroblock.

2. THE ALGORITHM

The proposed motion estimation algorithm belongs to the class of the spatial-temporal correlation techniques, which are based on the similarities between neighboring motion vectors belonging to the same frame (spatial correlation) or to adjacent frames (temporal correlation). These algorithms tend to follow with great accuracy the real movement of a video sequence, and they produce well-aligned motion fields, particularly suitable for the differential coding scheme used by the Mpeg encoders family.

The proposed algorithm operates in two steps. The first one is the identification of the best motion vector within a set of candidates, called "predictors", chosen among the available spatially and temporally correlated motion vectors, so exploiting the results of the motion estimation

already performed on previous macroblocks. To reduce the computational complexity, only the neighboring spatial and temporal vectors are tested, and among them the best one is selected.

The cost function used to evaluate the goodness of a predictor is the Sum of Absolute Differences (SAD) between the pixel values of the predictor and of the current macroblock (CMB) under estimation. The SAD was chosen instead of the Mean Square Error, because of the good reliability of the results and the less computation power requested.

If the macroblock is a square region of 16 by 16 pixels, the SAD is defined in this way: let $SAD(x,y)$ be the SAD between a macroblock in the position (x, y) in the reference n frame, having pixels of intensity $P_n(x+i, y+j)$, and a corresponding macroblock in the position $(x+dx, y+dy)$ in the m frame, having pixels of intensity $P_m(x+dx+i, y+dy+j)$, then

$$SAD(x,y) = \sum_{i=0}^{15} \sum_{j=0}^{15} |P_n(x+i, y+j) - P_m(x+dx+i, y+dy+j)|$$

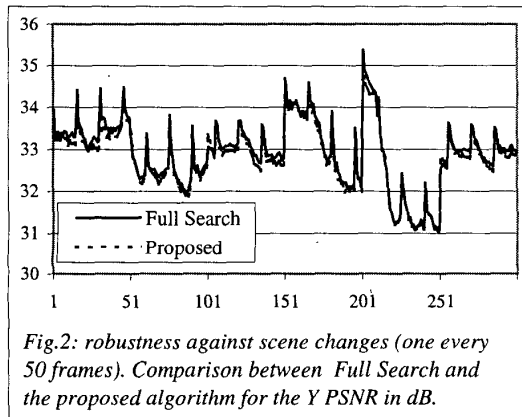


Fig.2: robustness against scene changes (one every 50 frames). Comparison between Full Search and the proposed algorithm for the Y PSNR in dB.

After choosing the best vector in the first step, a refinement step is then applied around the position pointed by the selected predictor, in search for the optimal motion vector. The motion vectors actually tested, called “updates”, depends adaptively on the motion amount contained in the video sequence under coding, while the number of vectors tested is fixed and predetermined for each macroblock, regardless of the search window range. For a typical search window size of $[-16;+16]$ pixels the computational complexity is reduced by 99% compared to the Full Search approach, and much more if the window is widened (see figure 1). This happens because the Full Search performs an exhaustive evaluation on all the motion vectors included in the search window, so its complexity depends on the square of the window side,

which makes it unfeasible in a low-power hardware. On the contrary, the fixed workload showed by our algorithm facilitates the design of an efficient hardware implementation, highly optimized for power consumption and silicon area.

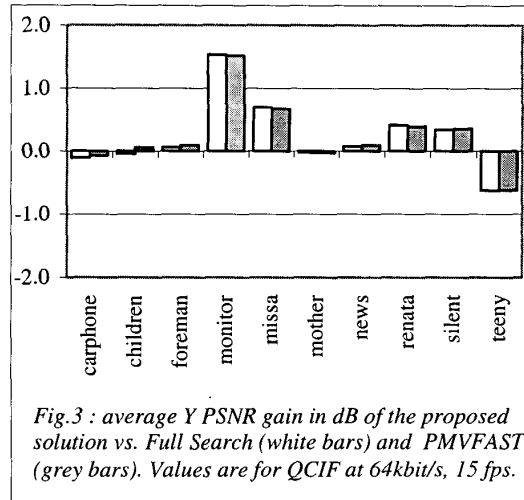


Fig.3 : average Y PSNR gain in dB of the proposed solution vs. Full Search (white bars) and PMVFAST (grey bars). Values are for QCIF at 64kbit/s, 15 fps.

2.1 Performances evaluation

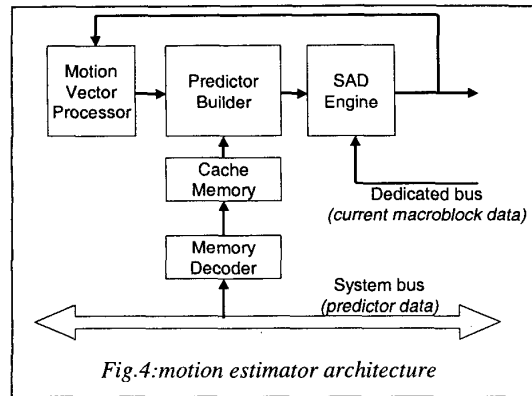
To prove its effectiveness, the proposed motion estimation algorithm has been included in a MPEG-4 compliant encoder developed in our labs, and compared with the standard Full Search approach on different video sequences, decoding the bitstreams using the official reference decoder developed by MoMuSys. Simulations proved that the quality provided by the proposed algorithm is comparable or better than the Full Search (see figure 6).

We also evaluated the robustness of the algorithm against scene changes. They are critical situations because whenever a scene change occurs the temporal correlation between motion vectors is completely lost, so the reference given by the temporal predictors could lead to wrong results and compromise the convergence of the motion estimation process. However, in the simulations performed the proposed algorithm demonstrates a robustness equivalent to the Full Search (see figure 2). The conclusion is that, for the given application, a scene change detection feature is not strictly needed.

The proposed algorithm has also been compared to the PMVFAST (Predictive Motion Vector Field Adaptive Search Technique), an optimized motion estimation algorithm adopted by ISO/IEC as reference benchmark for commercial MPEG-4 compliant products, and included as recommended tool in the official MPEG-4 encoder software [26]. The PMVFAST exploits the spatial correlation among adjoining motion vectors to choose the search start center, and then performs a

recursive diamond search until a good enough vector is found. The simulations performed demonstrates that our method can achieve better results than PMVFAST with an average 50% less computational workload (see figure 3), considering that PMVFAST has variable computational complexity.

3. THE ARCHITECTURE



The system architecture of the proposed motion estimator coprocessor, shown in figure 4, is composed by five main blocks.

The control stage is the Motion Vector Processor, a programmable application specific core designed to run motion estimation algorithms. The Predictor Builder receives the images data from the frame buffer through a local Cache Memory, and produces the final 16x16 predictor. The data contained in the frame buffer are compressed to save silicon area, power and bandwidth, and are decompressed by the Memory Decoder before entering the Cache. The SAD engine computes the matching error between the CMB and the current predictor. All these blocks will be described in detail in the next paragraphs.

It is worth saying that the motion estimator system works internally in a pipelined fashion, which allows a very efficient use of the available hardware resources per time unit. Moreover, the upper system layer can program some parameters of the estimator: i.e. the memory compression feature and the unrestricted motion vectors can be enabled or disabled by simply writing a configuration register.

3.1 Architecture exploration

To evaluate the hardware parameters of the system, a software model of the architecture has been written in Ansi-C language and included in our proprietary MPEG-4 encoder. The architectural software model provides exactly the same results of the purely algorithmic model previously used to assess the quality performances, but in

addition it reflects the system organization discussed above, and it can be configured with parameters like clock frequency, cache characteristic, bus width and SAD array size. In return, the architectural model can measure cache efficiency – for instance (but not only) in terms of hit/miss ratio – the workload sustained by the different blocks of the system, the bandwidth required on the system bus, and so on. In such a way it was possible to evaluate the best trade-off in terms of hardware complexity versus the computational load and power efficiency.

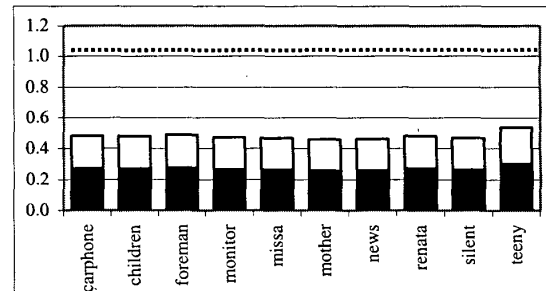


Fig.5: average bandwidth in MB/s without (white bars) and with memory compression (black bars) for QCIF at 15 fps. The dashed line is the bandwidth without cache and without memory compression.

The architectural model was also imported in CoWare N2C [27] and simulated at Bus Cycle Accurate level.

In summary, with the selected hardware configuration, the proposed motion estimator system has a die area smaller than a square millimeter in 0.18 μm technology, and it needs less than 2MHz clock frequency for QCIF encoding at 15 fps, and less than 7 MHz for CIF at the same rate.

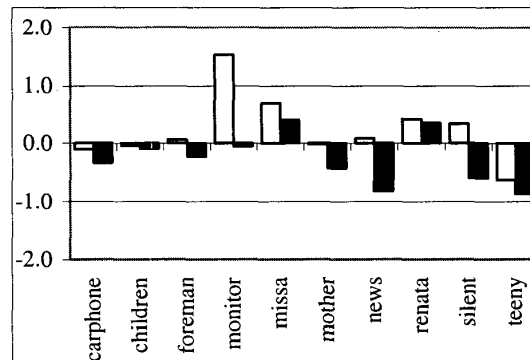


Fig.6: average Y PSNR gain in dB of the proposed solution vs. the Full Search, without (white bars) and with memory compression (black bars). Values are for QCIF at 64kbit/s, 15 fps.

3.2 Motion Vector Processor

In general, the controller part of a motion estimator can be implemented in hardware or in software. In the first option, the controller is a hardwired FSM (Finite State Machine), which ensures high efficiency, small silicon area and low-power operation but having, as a drawback, no flexibility at all. The alternative is a pure software implementation, which means that the controller is executed on a dedicated CPU core. This provides the maximum flexibility, as the CPU is fully programmable, but at the same time it could be very slow and power-consuming, if compared to the hardware solution.

Our Motion Vector Processor (MVP) has a hardware complexity comparable to a hardwired FSM explicitly designed to run a specific motion estimation algorithm but, being programmable, it has flexibility comparable to a general-purpose CPU, from the motion estimation point of view, and can be easily reprogrammed to target different applications and video encoding standards.

The instruction set of the MVP allows to generate different kind of motion vectors - i.e. temporal/spatial predictors and fixed vectors -, to vary the program flux with conditional jumping, to store or discard vectors, et cetera. Each instruction is expressed on a 24 bits word, and the chosen motion estimation algorithm is encoded using only 27 instructions, for a total memory occupancy of 81 bytes. Other motion estimation algorithms can be easily loaded and run on the MVP. Furthermore, the MVP generates a control word used to configure the behavior of the other blocks of the motion estimator.

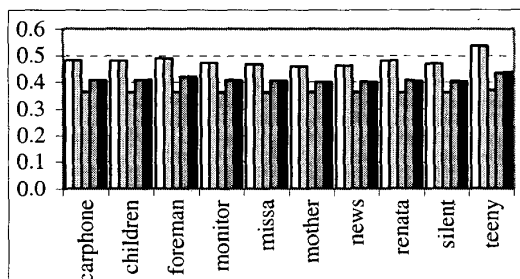


Fig.7: average bandwidth in MByte/s for different cache organizations (2kB 1way, 2kB 2way, 4kB 1way, 4kB 2way, 4kB 4way), 15fps QCIF.

3.3 Cache Memory

The predictor data coming from the frame buffer are stored locally in a Cache Memory. The cached approach to the memory access allows a great bandwidth reduction on the system bus, and minimizes the conflicts of the motion estimator with the other blocks of the system. Furthermore, thanks to the search range independency of the algorithm, it allows to widen the search window, increasing the quality performances while maintaining a

fixed hardware complexity in terms of local memory size, bandwidth required on the system bus and computational power needed (see figure 7). A set of simulations has been performed to select the best cache organization in terms of size, associativity, line organization and refill policy. A simple 2 kByte, 2-way set associative cache with FIFO refill policy and a cache line size of 32 Bytes resulted enough for QCIF and CIF video encoding. This configuration provides a mean bandwidth reduction of nearly 50%, but nothing is guaranteed on the peak bandwidth, which can become much higher than the average one, causing possible conflicts on the main bus, as the motion estimator demands more and more resources to the system (see figure 8). For this reason, and also to further limit the power consumption, we introduced a feature to cut the peak bandwidth to a certain amount, allowing only a fixed and programmable number of cache refills during a macroblock period. The effect on the algorithm is that some predictors can be lost because the cache controller does not allow to fetch their data from the main memory.

Simulations showed that the average quality loss caused by the bandwidth limitation is in the order of 1/100th of dB in terms of PSNR (Peak Signal to Noise Ratio) when the peak bandwidth is halved.

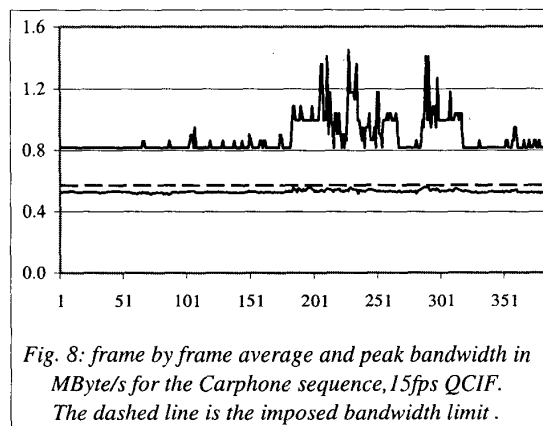


Fig. 8: frame by frame average and peak bandwidth in MByte/s for the Carphone sequence, 15fps QCIF. The dashed line is the imposed bandwidth limit.

3.4 Memory Decoder

The bandwidth, and therefore the switching activity, is further reduced thanks to a lossy data compression algorithm [28]: the data are stored in the frame buffer in a compressed form, and are decoded before entering the cache. The algorithm adaptively operates on luminance and chrominance samples, achieving a fixed compression ratio of nearly 50%.

Because the power consumption on a bus is proportional to the switching activity, intended as the number of commutations per time unit, the memory compression

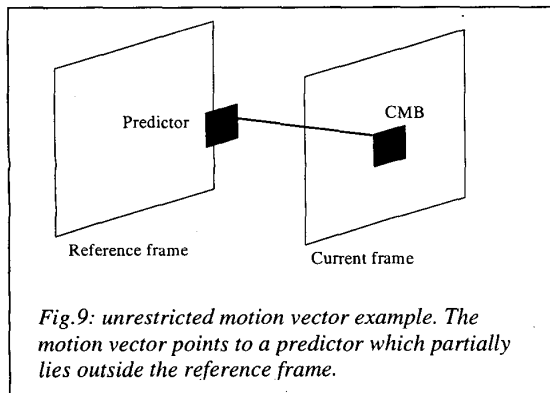
allows to halve both the buffer size and the power consumption on the bus at the same time.

It is worth saying that the memory co-decompression feature does not introduce any kind of mismatch between the MPEG-4 encoder and decoder, and it guarantees a negligible video quality loss (see figure 6).

3.5 Predictor Builder

The predictor data coming from the cache must be manipulated in order to produce the final 16 by 16 pixels predictor. First of all, the pixel data are converted from the block-scan organization to the conventional raster-scan fashion. Then, the sub-pixel interpolation is applied. To do that, a 17 by 17 area must be fetched from the memory, and then a simple linear average filter is applied to compute the half-pixel positions. The MPEG-4 allows also the unrestricted motion vectors: this means that a vector can point to a predictor which partially lies outside the frame (see figure 9). In this case, a border extension must be performed to build the external part of the predictor, by means of a simple replica of the innermost pixels value. This task is assigned to the Predictor Builder.

As previously said, the bandwidth limitation feature of the motion estimator could cause a loosing of some of the data belonging the predictor under estimation. In this case, the Predictor Builder must discard the current predictor, which will be ignored.



3.6 SAD Engine

This stage computes the matching error between the CMB and the predictor. In the proposed system, the CMB data is received directly from the sensor through a dedicated bus, while the predictor data comes from the Predictor Builder stage. The SAD engine is composed by a very small array of SAD units, which compute the matching error on a pixel-by-pixel basis, accumulating locally the partial results until the overall computation has finished.

This approach avoids the need of large systolic arrays, which are expensive on both silicon area and power consumption. We selected the best trade-off between the maximum number of SADs computed in parallel, and the overall clock frequency of the motion estimator system, choosing a SAD array of eight units. In other words, eight pixels of both the predictor and CMB data are processed in parallel in one clock cycle. The matching error is returned to the Motion Vector Processor, which can use it to evaluate the goodness of the current motion vector during the motion estimation phase.

4. CONCLUSIONS

The proposed algorithm and architecture for MPEG-4 low-bitrate encoding achieves a quality comparable or superior than the common Full Search approach, as well as the optimized PMVFAST algorithm, with remarkable savings in terms of number of operations performed, internal and external memory requirements, silicon area and bandwidth occupancy on the system bus. The ultra-low power consumption makes it an ideal solution for next generation mobile devices.

5. ACKNOWLEDGEMENTS

The authors gratefully acknowledge their colleagues in the Agrate Lab. of Advanced System Technology for the precious support: Daniele Bagni, Sara Bocchio, Massimo Mancuso, Luca Pezzoni, Emiliano Piccinelli, Alberto Rosti, Amedeo Zuccaro. We also thank Antonio Chimienti of the Italian National Research Council, Turin, for his valuable contributions.

6. BIBLIOGRAPHY

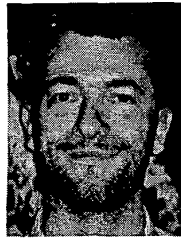
- [1] ISO/IEC 14496-2, "Information Technology - Generic coding of audio-visual objects", Part 2: Visual.
- [2] 3GPP TS 23.140, "GSM/UMTS Multimedia Messaging Service functional description".
- [3] M.A.Elgameh, A.M.Shams, M.A.Bayoumi, "A comparative analysis for low power motion estimation VLSI architectures", IEEE workshop on signal processing systems, 2000.
- [4] S.Kittitornkun, Y.H.Hu, "Frame-level pipelined motion estimation array processor", IEEE transactions on circuits and systems for video technology, Vol.11, N.2, February 2001.
- [5] J.-F.Shen, L.G.Chen, H.C.Chang, T.C.Wang, "Low-power full search block matching motion estimation chip for H.263+", ISCAS '99 conference proceedings, Vol.4.
- [6] L.Sousa, N.Roma, "Low-power array architectures for motion estimation", IEEE third workshop on multimedia signal processing, 1999.

- [7] V.L.Do, K.Y.Yun, "A low power VLSI architecture for full-search block-matching motion estimation", IEEE transactions on circuits and systems for video technology, Vol.8, N.4, August 1998.
- [8] H.Yeo, Y.H.Hu, "A novel matching criterion and low power architecture for real-time block based motion estimation", Proceedings of international conference on application specific systems, architectures and processors, 1996.
- [9] X-D.Zhang, C-Y.Tsui, "An efficient and reconfigurable VLSI architecture for different block matching motion estimation algorithms", Proceedings of the international conference on acoustic, speech and signal processing, 1997.
- [10] T-H.Chen, "A cost-effective three-step hierarchical search block-matching chip for motion estimation", IEEE journal of solid state circuits, Vol.33, N.8, August 1998.
- [11] R.S.Richmond II, D.S.Ha, "A low power motion estimation block for low bitrate wireless video", International symposium on low power electronics and design, 2001.
- [12] H.Yeo, "A modular high-throughput architecture for logarithmic search block-matching motion estimation", IEEE transactions on circuits and systems for video technology, Vol.8 N.3, June 1998.
- [13] Y.S.Seo, J.H.You, "A VLSI design of hierarchical search motion estimation processor chip", The first IEEE Asia Pacific conference on ASICs, 1999.
- [14] J.Minocha, N.R.Shanbhag, "A low power data-adaptive motion estimation algorithm", IEEE 3rd workshop on multimedia signal processing, 1999.
- [15] S.Zhu, K-K.Ma, "A new diamond search algorithm for fast block-matching motion estimation", IEEE transactions on image processing, Vol.9, N.2, February 2000.
- [16] J.Ju, Y-K.Chen, S.Y.Kung, "A fast rate-optimized motion estimation algorithm for low-bitrate video coding", IEEE transactions on circuits and systems for video technology, Vol.9, N.7, October 1999.
- [17] Y-T.Roan, P-Y.Chen "A fuzzy search algorithm for the estimation of motion vectors", IEEE transactions on broadcasting, Vol.46, N.2, June 2000.
- [18] S.Li, W-P.Xu, H.Wang, N-N.Zheng, "A novel fast motion estimation method based on genetic algorithm", Proceedings of the international conference on image processing, 1999.
- [19] K.T.Choi, S.C.Chan, T.S.Ng, "A new fast motion estimation algorithm using hexagonal subsampling pattern and multiple candidate search", Proceedings of the international conference on image processing, 1996.
- [20] Z-L.He, C-Y.Tsui, K-K.Chan, M.L.Liou, "Low-power VLSI design for motion estimation using adaptive pixel truncation", IEEE Transactions on circuits and systems for video technology, Vol.10, N.5, August 2000.
- [21] G.de Haan, P.Biezen, H.Huijgen, O.Ojo, "True motion estimation with 3-D recursive search block matching", IEEE transactions on circuits and systems for video technology, Vol.3, N.5, October 1993.
- [22] F.Rovati, D.Pau, E.Piccinelli, L.Pezzoni, J.-M. Bard, "An innovative, high-quality and search window independent motion estimation algorithm and architecture for Mpeg-2 encoding", IEEE transactions on consumer electronics, August 2000, Vol.46, N.3.
- [23] F.Momers, S.Dogimont, "A limited hardware resources efficient motion estimation algorithm", Proceedings of the IEEE international symposium on circuits and systems, Vol.4, 2000.
- [24] C.De Vleeschouwer, T.Nilsson, "Motion estimation for low-power video devices", IEEE international conference on image processing, 2001.
- [25] S-H.Han, S-W.Kwon, T-Y.Lee, M-K. Lee, "Low power motion estimation algorithm based on temporal correlation and its architecture", Sixth International Symposium on Signal Processing and its Applications, 2001.
- [26] ISO/IEC 14496-7, "Information Technology – Generic coding of audio-visual objects", Part 7: optimised reference software for coding of audio-visual objects, version 4.0, December 2001.
- [27] I.Bolsens, H.De Man, B.Lin C.Van Rompaey, S.Vercateren, D.Verkest, "Hardware/software co-design of digital telecommunication systems", Proceedings of the IEEE, Vol.85, N.3, March 1997.
- [28] A.Chimienti, M.Lucenteforte, D.Pau, R.Sannino, "A novel co-decoding scheme to reduce memory in Mpeg-2 MP@ML decoder", International Symposium on Signals Systems and Electronics, 1998.



Daniele ALFONSO was born in Alghero, Italy, in 1972. In 1998 he received a master degree in Electrical Engineering from the "Politecnico di Torino" university, and then he joined STMicroelectronics, Advanced System Technology group, working jointly with the Research Institute on Computer and Telecommunication Engineering of the Italian National Research Council. Later, he started working on moving pictures encoding and transcoding (H.263, Mpeg-2, Mpeg-4, H.26L), and particularly on low-power motion estimation for Mpeg-4. His main interests are algorithms

and architectures for digital video applications and he holds several patents granted in Europe.



Fabrizio S. ROVATI was born in Monza, Italy in 1971. He received electronic engineering degree at the Milan Polytechnic, Italy, in 1996. He joined STMicroelectronics Ltd., Bristol, UK (formerly INMOS Ltd.) where he contributed to the development of an MPEG-2 transport demultiplexer co-processor. He then joined STMicroelectronics' Advanced System Technologies in 1998 where he worked on an MPEG-2 motion estimation co-processor and on MPEG video encoders system architectures. He published four papers and holds seven patents and nine patent applications in digital video signal processing and architectures field. He is contract professor at Pavia Polytechnic University and gave several lectures at Milan Polytechnic University. His main interests are in digital video signal processing and related system-level and processors architectures.



Danilo Pietro PAU was born in Milan, Italy in 1966 and received a degree in electronic engineering from the Politecnico di Milano, Italy, in 1992. Started to work in SGS-THOMSON as stage from late 1991 to 1993, he worked at Corporate Advanced System Architectures laboratory in the Consumer group based in Agrate Brianza on High Definition Digital Video Decoder Reduced Memory design (HD-MAC). Since late 1994, after military service, he joined Terminals Strategic Unit of SGS-THOMSON as system engineer, working on Set Top Box application, then on Memory Reduction algorithm development and VHDL modeling for MPEG-2 video decoder. Since 1998 he is inside Advanced System technology - Digital Video Technology group. His main interests in the field of Digital Video Signal Processing are: Motion Estimation for Mpeg coding, Video transcoding, non-standard video coding where co-authored some papers and patents. In the field of Digital Architectures his interests are: programmable Architectures for Ultra Low Power Systems, ILP and TLP processing architectures.

Luca CELETTO was born in Udine, Italy, in 1972. In 1998 he received a master degree in Electrical Engineering from Padua University, then he joined STMicroelectronics, Advanced System Technology group, working on moving pictures encoding (H.263, Mpeg-2, Mpeg-4, H.26L). His main interests are algorithms and architectures for digital video applications.

